PHASE 2 — DEEP DIVE

Backend Engineering for AI Products

Month 3–5 · Every resource with exact video title + channel name

AI Product Engineer Master Roadmap — Phase 2 Companion Document

Phase Duration	12 weeks (Month 3, 4, and 5 of the full roadmap)
Daily Commitment	2–3 hours weekdays, 4–6 hours weekends
Your Starting Point	You finished Phase 1 — AI streaming chat app is live on Vercel. Now add the backend.
Goal of This Phase	Build a production-grade backend: API server, databases, auth, queues, billing, system design
Why It Matters for AI	AI products need auth for multi-tenancy, Redis to cache LLM responses, queues for async AI tasks, and Stripe for usage-based billing
Total Topics	10 core topics — Node.js, Fastify, PostgreSQL, Redis, Auth, SSE, Queues, Security, System Design, Stripe
End State	Production-ready AI SaaS backend — open-sourced on GitHub as 'ai-saas-starter'
Capstone Project	AI SaaS Starter Platform — Portfolio Project #2

TOPIC 1 — Node.js Runtime & Internals · *Weeks 1–2 · Know the engine before building the car*

You cannot build reliable AI backends without understanding Node.js deeply. AI processing tasks involve heavy I/O (file uploads, API calls, database writes), streaming responses, and long-running operations. Node.js has specific patterns for all of these — and specific failure modes when you get them wrong.

1.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	Node.js Full Course for Beginners | Complete All-in-One Tutorial (8.5 hrs)	Dave Gray	YouTube → Search 'Node.js Full Course Dave Gray Complete All-in-One'	The most comprehensive Node.js course on YouTube. Covers everything from scratch to deployment. Start here.
2	Playlist	Node.js Tutorial for Beginners (playlist, 12 videos)	The Net Ninja	YouTube → Search 'Node.js Tutorial for Beginners The Net Ninja' → open playlist	Excellent supplementary series — short focused videos. Watch alongside Dave Gray for clarity.
3	Video	Node.js Event Loop — Everything You Need to Know	Hussein Nasser	YouTube → Search 'Node.js Event Loop Hussein Nasser Everything You Need to Know'	The best deep-dive into Node.js internals on YouTube. Hussein explains libuv, thread pool, and I/O.
4	Video	Node.js Streams — The Complete Guide	Hussein Nasser	YouTube → Search 'Node.js Streams Complete Guide Hussein Nasser'	Streams are fundamental for AI file processing and LLM streaming. Must watch.
5	Video	Node.js Cluster Module & Worker Threads Explained	Hussein Nasser	YouTube → Search 'Node.js Cluster Module Worker Threads Hussein Nasser'	How to use multiple CPU cores in Node.js — needed for CPU-intensive AI tasks.
6	Video	Node.js Performance — How to Handle 10,000 Requests	Fireship	YouTube → Search 'Node.js Performance Handle 10000 Requests Fireship'	Practical performance tips. Sets context for why Node.js architecture choices matter.
7	Video	Building HTTP Server from Scratch in Node.js	Traversy Media	YouTube → Search 'Building HTTP Server Scratch Node.js Traversy Media'	Build a raw HTTP server without Express. Forces understanding of HTTP at the protocol level.
8	Docs	Node.js Official Documentation — Guides	Node.js Team	nodejs.org/en/docs/guides — read: Event Loop, Timers, Don't Block the Event Loop	Essential reading. 'Don't Block the Event Loop' is the most important Node.js article ever written.

1.2 — What to Study

Node.js Architecture & libuv

- libuv — the C library that gives Node.js async I/O and the thread pool
- Node.js event loop phases: timers → pending callbacks → idle → poll → check → close
- Thread pool (4 default threads) — for crypto, DNS, file system operations
- Why CPU-intensive operations (like local model inference) BLOCK the event loop
- Solution: worker_threads for CPU work, cluster for multi-core web servers

Streams — Critical for AI File Processing

- Readable, Writable, Duplex, Transform streams
- pipe() and pipeline() — the correct way to chain streams
- Backpressure — what it is and why ignoring it causes memory crashes
- Practical: stream a large PDF file through a parser without loading it into memory
- Practical: stream LLM tokens from OpenAI API through your server to the client

> ⍰ AI apps process large files (PDFs, audio, video) constantly. If you load a 100MB PDF into memory to process it, your Node.js server will crash under load. Streams prevent this.

Modules — CommonJS vs ESM

- CommonJS: require(), module.exports — how Node.js has worked for 15 years
- ESM: import/export — the modern standard, now supported in Node.js natively
- When to use each — mixing CJS and ESM causes subtle bugs
- package.json 'type: module' field — enables ESM by default

Error Handling Patterns

- Uncaught exceptions vs unhandled Promise rejections — both crash Node.js
- process.on('uncaughtException') and process.on('unhandledRejection') — safety nets
- Graceful shutdown — drain connections before killing the process
- AI-specific: handle OpenAI API timeout errors without crashing the server

> ⍰ Rule: Every async function in your AI backend must have a try/catch. Every Promise must have a .catch(). One unhandled rejection in production will kill your server if you don't handle it.

1.3 — Stop Conditions

☐ Build a raw HTTP/1.1 server in Node.js (no Express/Fastify) that handles JSON POST requests
☐ Implement a streaming file processor: read a 50MB PDF in chunks, process each chunk, write output — never loading the full file into memory
☐ Explain Node.js event loop phases and demonstrate that CPU work blocks I/O (with a timed benchmark)
☐ Implement graceful shutdown: on SIGTERM, finish in-flight AI requests before closing
☐ Use worker_threads to offload a CPU-intensive task (e.g., text parsing) off the main thread

- ☐ Configure Node.js cluster to use all available CPU cores on a multicore machine

TOPIC 2 — REST API Design with Fastify · *Weeks 2–3 · 2–3x faster than Express; built for AI backends*

Fastify is the production-grade Node.js web framework. It is 2–3x faster than Express, has first-class TypeScript support, built-in schema validation with JSON Schema, and a plugin architecture that scales cleanly. All AI SaaS backends in this roadmap use Fastify.

2.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Fastify — The Fastest Node.js Web Framework (Full Tutorial)	Traversy Media	YouTube → Search 'Fastify Fastest Node.js Web Framework Full Tutorial Traversy Media'	Best practical intro to Fastify. Covers routes, plugins, hooks, schema validation.
2	Video	Fastify vs Express — Why I Switched for Production	Fireship	YouTube → Search 'Fastify vs Express Why I Switched Fireship'	10-minute comparison video. Sets context for why Fastify is the right choice.
3	Video	REST API Design Best Practices — Full Course (2 hrs)	Web Dev Simplified	YouTube → Search 'REST API Design Best Practices Full Course Web Dev Simplified'	Best REST API design course. Covers versioning, error formats, status codes, pagination.
4	Video	Build a REST API with Fastify and TypeScript	Marius Espejo	YouTube → Search 'REST API Fastify TypeScript Marius Espejo'	Production setup: Fastify + TypeScript + Zod validation. Exactly the stack for this roadmap.
5	Video	OpenAPI / Swagger Documentation with Fastify	Simon Plenderleith	YouTube → Search 'OpenAPI Swagger Fastify Simon Plenderleith'	Auto-generate API docs from your Fastify routes — professional-grade API documentation.
6	Video	Fastify Plugins — How to Structure a Large Application	Matteo Collina (Fastify creator)	YouTube → Search 'Fastify Plugins Structure Large Application Matteo Collina'	From the creator of Fastify. The definitive guide to Fastify's plugin architecture.
7	Video	API Versioning — How to Do It Right	Hussein Nasser	YouTube → Search 'API Versioning How to Do It Right Hussein Nasser'	Versioning strategy for AI APIs that will evolve over time.
8	Docs	Fastify Official Documentation	Fastify Team	fastify.dev/docs — read: Getting Started, Routes, Hooks, Plugins, Schema Validation	Best framework docs in the Node.js ecosystem. Read the full 'Getting Started' and 'Reference' sections.

2.2 — What to Study

Fastify Core Architecture

- Route definition: GET, POST, PUT, PATCH, DELETE with typed request/reply
- Plugin system: encapsulated plugins for auth, database, AI providers
- Hooks: onRequest, preHandler, onSend, onClose — middleware done right
- Schema validation: JSON Schema for request body, querystring, params, response
- Auto-generated TypeScript types from JSON Schema — zero manual typing

REST API Design Principles

- Resource naming: nouns not verbs — /conversations not /getConversations
- HTTP methods semantic use: GET (read), POST (create), PUT (replace), PATCH (update), DELETE
- Status codes: 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, 500
- Consistent error response format: { error: string, message: string, statusCode: number }
- API versioning: /api/v1/... — always version from the start
- Pagination: cursor-based (for AI conversation streams) vs offset-based

AI-Specific API Patterns

- POST /api/v1/chat — accept a message, return a streaming SSE response
- POST /api/v1/documents — accept multipart file upload, return job ID
- GET /api/v1/jobs/:id — polling endpoint for async AI task status
- Idempotency keys — prevent duplicate AI job submissions

> ⍰ Design your AI API to be provider-agnostic. The route should accept { model: 'gpt-4o' | 'claude-3-5-sonnet' } and your server routes to the right provider. Never expose 'openai_api_key' concerns to the client.

2.3 — Stop Conditions

☐ Build a fully typed Fastify REST API with Zod + JSON Schema validation — zero runtime type errors
☐ Implement consistent error handling: all errors return the same JSON schema regardless of cause
☐ Auto-generate OpenAPI/Swagger documentation from your route schemas
☐ Implement API versioning: /api/v1/ routes work independently of future /api/v2/ routes
☐ Build a Fastify plugin that registers the database connection and makes it available to all routes
☐ Implement cursor-based pagination for a conversation history endpoint
☐ Write integration tests for all API routes using Fastify's inject() method — no real HTTP calls needed

TOPIC 3 — PostgreSQL & Prisma ORM · *Weeks 3–4 · The database layer of every serious AI SaaS*

PostgreSQL is the database of choice for AI SaaS products. It stores user data, conversation history, document metadata, AI job results, and with pgvector it can store embeddings too. Prisma gives you type-safe database access with automatic migrations. Learn both deeply.

3.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	PostgreSQL Tutorial Full Course 2022 (4.5 hrs)	freeCodeCamp.org	YouTube → Search 'PostgreSQL Tutorial Full Course 2022 freeCodeCamp'	Most comprehensive free PostgreSQL course. Covers SQL fundamentals, joins, indexes, transactions.
2	Video	Prisma ORM — Complete Tutorial (2.5 hrs)	Lama Dev	YouTube → Search 'Prisma ORM Complete Tutorial Lama Dev'	Best Prisma tutorial. Covers schema, migrations, CRUD, relations, transactions.
3	Video	Prisma with Fastify — Full CRUD API Tutorial	Marius Espejo	YouTube → Search 'Prisma Fastify Full CRUD API Tutorial Marius Espejo'	Combines Phase 2 stack: Fastify + Prisma. Build it alongside watching.
4	Video	PostgreSQL Indexes — How They Actually Work	Hussein Nasser	YouTube → Search 'PostgreSQL Indexes How They Actually Work Hussein Nasser'	The most important database performance topic. Every AI SaaS needs proper indexing.
5	Video	PostgreSQL EXPLAIN ANALYZE — Query Performance Tuning	Hussein Nasser	YouTube → Search 'PostgreSQL EXPLAIN ANALYZE Query Performance Hussein Nasser'	How to read PostgreSQL query plans and optimise slow queries.
6	Video	Database Schema Design for SaaS Applications	Theo (t3.gg)	YouTube → Search 'Database Schema Design SaaS Applications Theo t3gg'	Multi-tenant SaaS schema patterns — organisations, teams, users, roles.
7	Video	PostgreSQL Full Text Search — Better Than You Think	Hussein Nasser	YouTube → Search 'PostgreSQL Full Text Search Hussein Nasser'	Full-text search is useful for AI apps before adding vector search.
8	Video	Prisma Transactions — How to Use Them Correctly	Prisma (official channel)	YouTube → Search 'Prisma Transactions How to Use Correctly Prisma official'	Transactions for atomic AI operations — e.g., create conversation + first message atomically.
9	Video	pgvector — Store and Query Embeddings in PostgreSQL	Supabase (official channel)	YouTube → Search 'pgvector Store Query Embeddings PostgreSQL Supabase'	Introduction to pgvector — store embeddings directly in Postgres for simple RAG setups.
10	Docs	Prisma Official Documentation	Prisma Team	prisma.io/docs — read: Getting Started, Schema, Client, Migrate	The gold standard of ORM documentation.

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
					Read the 'Concepts' section thoroughly.

3.2 — What to Study

PostgreSQL Fundamentals

- Data types: TEXT, VARCHAR, INTEGER, BIGINT, BOOLEAN, JSONB, TIMESTAMPTZ, UUID
- DDL: CREATE TABLE, ALTER TABLE, DROP TABLE, indexes
- DML: SELECT, INSERT, UPDATE, DELETE
- Joins: INNER, LEFT, RIGHT, FULL OUTER — know when to use each
- Aggregations: GROUP BY, HAVING, COUNT, SUM, AVG, window functions
- CTEs (Common Table Expressions): WITH clauses for complex queries

Schema Design for AI SaaS (Multi-Tenant)

- Organisations table: id, name, stripe_customer_id, plan_tier, created_at
- Users table: id, org_id (FK), email, role, created_at
- Workspaces table: id, org_id (FK), name — logical grouping for conversations
- Conversations table: id, workspace_id (FK), title, model, system_prompt, token_usage
- Messages table: id, conversation_id (FK), role, content, token_count, latency_ms
- AI_jobs table: id, org_id, type, status, payload (JSONB), result (JSONB), created_at
- Usage_records table: id, org_id, tokens_used, cost_usd, model, created_at

> ⍰ Add 'org_id' to every table that should be tenant-isolated. Add a Row Level Security (RLS) policy in PostgreSQL — even if your app handles tenant isolation, RLS is a safety net against bugs.

Indexes — Performance for AI Query Patterns

- B-tree index: default — used for equality and range queries
- Index on conversation_id in messages table — critical for conversation history queries
- Composite index: (org_id, created_at) for tenant-filtered time-series queries
- Partial index: WHERE status = 'pending' on ai_jobs table — only index relevant rows
- EXPLAIN ANALYZE: always check query plans for slow operations

Prisma Schema & Migrations

- schema.prisma: define models, relations, field types
- prisma migrate dev: create and apply migration files (development)
- prisma migrate deploy: apply migrations in production (CI/CD)
- Prisma Client: type-safe queries auto-generated from your schema
- Relations: @relation, one-to-many, many-to-many with explicit join tables
- Prisma transactions: $transaction() for atomic AI operations

> ⍰ Always use UUIDs (gen_random_uuid()) not sequential integers for IDs. Sequential IDs are predictable — a user could guess other users' conversation IDs. UUIDs are safe for public URLs.

3.3 — PostgreSQL + Prisma Mastery Schema Example

```
// prisma/schema.prisma — AI SaaS multi-tenant schema

model Organization {
  id              String        @id @default(uuid())
  name            String
  stripeCustomerId String?      @unique
  planTier        PlanTier      @default(FREE)
  users           User[]
  workspaces      Workspace[]
  usageRecords    UsageRecord[]
  createdAt       DateTime      @default(now())
}

model Conversation {
  id           String    @id @default(uuid())
  workspaceId  String
  workspace    Workspace @relation(fields:[workspaceId], references:[id])
  title        String    @default('New Conversation')
  model        String    @default('gpt-4o')
  systemPrompt String?
  tokenUsage   Int       @default(0)
  messages     Message[]
  createdAt    DateTime  @default(now())
  updatedAt    DateTime  @updatedAt
  @@index([workspaceId, createdAt])
}
```

3.4 — Stop Conditions

- ☐ Design and implement the full multi-tenant AI SaaS schema (7 tables) using Prisma schema
- ☐ Write a migration that adds an index to the messages table without downtime
- ☐ Query: find all conversations for an org in the last 7 days with their message count — use EXPLAIN ANALYZE
- ☐ Implement a Prisma transaction: create conversation + first message + usage record atomically
- ☐ Set up Row Level Security on the conversations table — users cannot access other orgs' data
- ☐ Add pgvector extension and store an embedding vector — query the top-5 nearest neighbours
- ☐ Write a query using window functions: rank conversations by token usage per workspace

TOPIC 4 — Redis: Caching, Sessions & Queues · *Week 5 · The biggest lever for cutting AI API costs*

Redis is not optional for AI SaaS. LLM API calls cost money — caching semantically similar queries saves 40–60% of API costs. Redis also handles session storage, pub/sub for real-time events, rate limiting counters, and queue metadata. Learn it deeply.

4.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Redis Crash Course — The Complete Tutorial (1.5 hrs)	Traversy Media	YouTube → Search 'Redis Crash Course Complete Tutorial Traversy Media'	Best comprehensive Redis tutorial. Covers all data types, commands, use cases.
2	Course	Redis for JavaScript Developers — Full Course	Redis University (official)	university.redis.com — free course 'Redis for JavaScript Developers'	Official Redis course. Free. Covers Node.js integration with ioredis.
3	Video	Redis in 100 Seconds	Fireship	YouTube → Search 'Redis in 100 Seconds Fireship'	Best 2-minute overview. Watch first to build the mental model.
4	Video	Caching Strategies Explained — Cache Aside, Write Through, Write Behind	Hussein Nasser	YouTube → Search 'Caching Strategies Explained Cache Aside Write Through Hussein Nasser'	Critical video. Covers all caching patterns — cache aside is the standard for AI response caching.
5	Video	Redis Pub/Sub — Real-Time Messaging in Node.js	Traversy Media	YouTube → Search 'Redis Pub Sub Real-Time Messaging Node.js Traversy Media'	Pub/Sub for broadcasting AI job completion events to connected clients.
6	Video	BullMQ — Redis-based Job Queues in Node.js (Full Tutorial)	Lama Dev	YouTube → Search 'BullMQ Redis Job Queue Node.js Full Tutorial Lama Dev'	BullMQ is the standard Node.js queue library. Uses Redis under the hood.
7	Video	Semantic Caching for LLM Applications — GPTCache Overview	Zep AI (official channel)	YouTube → Search 'Semantic Caching LLM Applications GPTCache Zep AI'	Semantic caching = cache LLM responses for similar (not just identical) queries. Game-changing for costs.
8	Video	Redis Rate Limiting — Sliding Window Algorithm	Hussein Nasser	YouTube → Search 'Redis Rate Limiting Sliding Window Algorithm Hussein Nasser'	Implement per-user, per-plan API rate limiting with Redis sorted sets.
9	Docs	ioredis Documentation (Node.js Redis client)	Luin (community)	github.com/redis/ioredis — README and API docs	The most reliable Node.js Redis client. Read the full API documentation.

4.2 — What to Study

Redis Data Structures — Know All Six

- STRING: key-value pairs — cache JSON, session tokens, feature flags
- HASH: field-value maps — store user session data as individual fields
- LIST: ordered sequences — simple message queues, conversation buffers
- SET: unique members — track which users are online, feature flag recipients
- SORTED SET (ZSET): scored members — rate limiting counters, leaderboards
- STREAM: append-only log — event sourcing for AI job events

LLM Response Caching — The #1 Cost Saver

- Exact cache: hash the (model + messages array) → store response → TTL 24h
- Semantic cache: embed the query → compare cosine similarity → return if >0.95 threshold
- Cache invalidation: clear cache when system prompt changes or user resets conversation
- Cache warming: pre-cache responses for common queries during off-peak hours

Implementation: Before calling OpenAI, compute SHA256 hash of (model + messages JSON). Check Redis with this key. If HIT, return cached response instantly — zero API cost, <1ms latency. If MISS, call OpenAI, store response in Redis with 24h TTL.

Rate Limiting with Redis (Sliding Window)

- Per-user rate limit: 20 requests/minute for free tier, 200 for pro tier
- Token-based rate limit: 10,000 tokens/day for free tier
- IP-based rate limit: prevent anonymous abuse before auth
- Redis MULTI/EXEC transactions for atomic counter operations

Session Storage

- Store JWT refresh tokens in Redis with matching TTL
- Store user preferences and model settings per session
- Invalidate all sessions for a user on password change
- Store streaming job progress — client polls this key during long AI operations

4.3 — Stop Conditions

☐ Implement exact-match LLM response caching — verify 0 OpenAI API calls on cache hits
☐ Implement sliding window rate limiting: 20 req/min per user using Redis ZADD and ZRANGEBYSCORE
☐ Implement per-plan token budget enforcement: block requests when daily token limit exceeded
☐ Use Redis Pub/Sub to notify clients when a background AI job completes
☐ Implement session storage: store user session in Redis with 7-day TTL, invalidate on logout
☐ Measure cache hit rate — achieve >40% cache hit rate on a simulated chat workload

TOPIC 5 — Authentication & Authorization · *Week 5–6 · Multi-tenant auth for AI SaaS*

Authentication is one of the most security-sensitive parts of any application, and AI SaaS adds extra complexity: you need multi-tenancy, API key auth for programmatic access, plan-based feature flags, and token usage enforcement per user. This topic covers all of it.

5.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	JWT Authentication — How It Works (Full Explanation)	Web Dev Simplified	YouTube → Search 'JWT Authentication How It Works Full Explanation Web Dev Simplified'	The definitive JWT tutorial. Understand every part of the token before using auth libraries.
2	Video	OAuth 2.0 and OpenID Connect Explained (Animated)	OktaDev	YouTube → Search 'OAuth 2.0 OpenID Connect Explained Animated OktaDev'	Best visual explanation of OAuth flows. Covers Authorization Code + PKCE flow used by Clerk/Auth.js.
3	Video	Clerk Authentication Full Tutorial — Next.js + Fastify	Jack Herrington	YouTube → Search 'Clerk Authentication Full Tutorial Next.js Jack Herrington'	Clerk is the fastest way to add auth to AI SaaS. Shows full integration with Next.js.
4	Video	Auth.js (NextAuth v5) Full Tutorial — App Router	Josh tried coding	YouTube → Search 'Auth.js NextAuth v5 Full Tutorial App Router Josh tried coding'	For understanding auth internals, Auth.js is better than Clerk. Learn both.
5	Video	API Key Authentication — How to Build It	Theo (t3.gg)	YouTube → Search 'API Key Authentication How to Build It Theo t3gg'	API keys for B2B programmatic access to your AI API — critical for developer-facing AI products.
6	Video	Role-Based Access Control (RBAC) in Node.js	Web Dev Simplified	YouTube → Search 'Role Based Access Control RBAC Node.js Web Dev Simplified'	Implement admin, member, viewer roles within an organisation.
7	Video	Refresh Tokens — How to Handle Them Securely	Hussein Nasser	YouTube → Search 'Refresh Tokens How to Handle Securely Hussein Nasser'	Secure refresh token rotation — prevents token theft in AI SaaS.
8	Video	Multi-Tenancy Authentication — Organisation-Based Auth	Theo (t3.gg)	YouTube → Search 'Multi-Tenancy Authentication Organisation Based Theo t3gg'	Exactly the auth pattern needed for AI SaaS — users belong to organisations.
9	Docs	Clerk Official Documentation	Clerk Team	clerk.com/docs — read: Next.js Quickstart, Backend API, Organisations	Clerk's docs are excellent. Read the Organisations section for multi-tenant setup.

5.2 — What to Study

JWT Internals

- JWT structure: Header.Payload.Signature — base64url encoded, not encrypted
- Signing algorithms: RS256 (asymmetric, use this) vs HS256 (symmetric)
- Access token (15 min TTL) + Refresh token (7 day TTL) pattern
- Token claims: sub (user ID), org_id, role, plan, iat (issued at), exp (expiry)
- NEVER store sensitive data in JWT payload — it is only base64 encoded, not encrypted

OAuth 2.0 + OpenID Connect Flows

- Authorization Code + PKCE flow — used for web apps (most secure)
- Client Credentials flow — used for server-to-server API communication
- Google, GitHub, Microsoft OAuth — the three providers to support
- OIDC id_token — contains verified user identity claims from the provider

Multi-Tenant Auth Pattern for AI SaaS

- User belongs to one or more Organisations
- Organisation has a plan_tier: FREE | STARTER | PRO | ENTERPRISE
- Every API request includes org_id context — enforced by middleware
- Feature flags: check org.plan_tier before allowing access to advanced AI models
- Usage enforcement: check org.monthly_token_usage before processing AI request

API Key Authentication for Developers

- Generate secure random API keys: crypto.randomBytes(32).toString('hex')
- Store HASHED keys in database (SHA256) — never store plaintext
- Show the key to user ONCE at creation — they must copy it immediately
- Key rotation: allow users to create new key, old key works for 24h grace period

> ⍰ Build your auth in layers: (1) Clerk for user-facing auth (fast setup), (2) Custom API key auth for B2B usage, (3) JWT verification middleware that accepts both. This covers all AI SaaS auth use cases.

5.3 — Stop Conditions

☐ Implement multi-tenant JWT auth: every API request validates org_id and plan_tier from JWT claims
☐ Implement API key authentication: generate, hash, store, and validate API keys for B2B access
☐ Build RBAC: org owner can invite members, set roles, remove members
☐ Implement feature-gating: only PRO plan users can access GPT-4o — enforced in Fastify middleware
☐ Implement usage enforcement: reject AI requests when monthly token limit is exceeded
☐ Secure refresh token rotation: new refresh token issued on each use, old one invalidated

TOPIC 6 — WebSockets & SSE for AI Token Streaming · *Week 6 · The protocol layer of LLM streaming*

Server-Sent Events (SSE) is the standard protocol for streaming LLM tokens from server to client. Understanding how it works at the HTTP level — not just through the Vercel AI SDK abstraction — makes you capable of building reliable, production-grade streaming with reconnection, backpressure, and error handling.

6.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Server-Sent Events (SSE) — The Ultimate Guide	Hussein Nasser	YouTube → Search 'Server-Sent Events SSE Ultimate Guide Hussein Nasser'	Best deep-dive on SSE. Covers the HTTP spec, reconnection, event IDs, and limitations.
2	Video	WebSockets vs Server-Sent Events vs Long Polling — When to Use Which	Hussein Nasser	YouTube → Search 'WebSockets vs Server-Sent Events vs Long Polling Hussein Nasser'	Definitive comparison. After this video you know exactly when to use SSE vs WebSockets.
3	Video	Building a Real-Time Chat App with WebSockets — Full Tutorial	Traversy Media	YouTube → Search 'Real-Time Chat App WebSockets Full Tutorial Traversy Media'	Build a WebSocket chat from scratch. Useful for understanding the protocol before using Socket.io.
4	Video	Socket.io Full Tutorial — Real-Time Node.js Applications	Lama Dev	YouTube → Search 'Socket.io Full Tutorial Real-Time Node.js Lama Dev'	When AI features need bidirectional real-time (collaborative editing, multiplayer AI).
5	Video	Implementing SSE in Fastify — Streaming AI Responses	Simon Plenderleith	YouTube → Search 'Implementing SSE Fastify Streaming Simon Plenderleith'	Exactly what you need: SSE in Fastify for LLM token streaming.
6	Video	Backpressure in Node.js Streams — How to Handle Slow Clients	Hussein Nasser	YouTube → Search 'Backpressure Node.js Streams Slow Clients Hussein Nasser'	If a client reads SSE slowly, your server must handle backpressure — Hussein explains it perfectly.
7	Docs	MDN — Using Server-Sent Events	Mozilla	developer.mozilla.org → search 'Using server-sent events'	The HTTP spec for SSE. Short, essential reading. Know the wire format: 'data: ... '

6.2 — What to Study

SSE Protocol Internals

- SSE is HTTP/1.1 with Content-Type: text/event-stream
- Wire format: data: <payload>\n\n — two newlines end each event
- Named events: event: token\ndata: hello\n\n

- Event IDs: id: 42\n — client sends Last-Event-ID header on reconnect
- Heartbeat: send a comment (': heartbeat\n\n') every 30s to prevent proxy timeouts

Production SSE for AI Streaming

```
// Fastify SSE endpoint for LLM token streaming
fastify.post('/api/v1/chat', async (request, reply) => {
  reply.raw.writeHead(200, {
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache',
    'Connection': 'keep-alive',
    'X-Accel-Buffering': 'no',  // disable nginx buffering
  });

  const stream = await openai.chat.completions.create({
    model: 'gpt-4o', messages: request.body.messages, stream: true
  });

  for await (const chunk of stream) {
    const token = chunk.choices[0]?.delta?.content || '';
    reply.raw.write(`data: ${JSON.stringify({ token })}\n\n`);
  }
  reply.raw.write('data: [DONE]\n\n');
  reply.raw.end();
});
```

Client Reconnection Handling

- EventSource API: browser auto-reconnects on disconnect after 3 seconds
- Last-Event-ID: client sends the last received event ID on reconnect
- Resume streaming: server resumes from the last event ID — requires event ID tracking
- Abort controller: cancel streaming when user navigates away

6.3 — Stop Conditions

☐ Build a raw SSE endpoint in Fastify — stream OpenAI tokens to the browser without any SDK
☐ Implement reconnection: assign event IDs, client reconnects and streaming resumes correctly
☐ Add heartbeat: send a comment event every 30s — verify Nginx/proxy doesn't close idle connections
☐ Handle abort: client sends abort request mid-stream, server cancels the OpenAI API call
☐ Load test: 500 concurrent SSE connections — server memory stays flat (no leak)

TOPIC 7 — Background Jobs & Async AI Pipelines · *Week 7 · Process documents, embeddings, and emails asynchronously*

Most AI operations should not run in the HTTP request-response cycle. Document parsing (30s), embedding generation (5s), email sending, and report generation are all async tasks. BullMQ gives you a Redis-backed job queue with retries, prioritisation, concurrency control, and a monitoring dashboard.

7.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	BullMQ — Complete Tutorial for Node.js (1.5 hrs)	Lama Dev	YouTube → Search 'BullMQ Complete Tutorial Node.js Lama Dev'	The best BullMQ tutorial. Covers queues, workers, processors, events, and retries.
2	Video	Background Jobs in Next.js and Node.js	Theo (t3.gg)	YouTube → Search 'Background Jobs Next.js Node.js Theo t3gg'	Architectural overview — when to use BullMQ vs Inngest vs direct async. Essential.
3	Video	Inngest — Serverless Background Jobs (Full Tutorial)	Inngest (official channel)	YouTube → Search 'Inngest Serverless Background Jobs Full Tutorial Inngest official'	Inngest is excellent for Next.js/Vercel deployments where BullMQ (Redis) is overkill.
4	Video	BullMQ — Job Queues with Redis in 60 Minutes	Anson the Developer	YouTube → Search 'BullMQ Job Queues Redis 60 Minutes Anson the Developer'	Fast-paced practical BullMQ tutorial — build a real job processing system.
5	Video	Building a Document Processing Pipeline — Node.js	Jack Herrington	YouTube → Search 'Document Processing Pipeline Node.js Jack Herrington'	Specifically covers AI document pipelines: upload → parse → chunk → embed → index.
6	Video	Idempotent Job Handlers — How to Prevent Duplicate Processing	Hussein Nasser	YouTube → Search 'Idempotent Job Handlers Duplicate Processing Hussein Nasser'	Critical pattern: if a job runs twice (crash + retry), it should produce the same result once.
7	Video	Bull Board — BullMQ Dashboard Setup	Kevin Kwok (community)	YouTube → Search 'Bull Board BullMQ Dashboard Setup'	@bull-board is the monitoring UI for BullMQ. Shows job status, failures, retry queue.
8	Docs	BullMQ Official Documentation	BullMQ Team	docs.bullmq.io — read: Quick Start, Patterns, Flows	Excellent docs. The 'Patterns' section covers idempotency, job groups, parent/child jobs.

7.2 — What to Study

BullMQ Core Concepts

- Queue: the named channel — 'document-processing', 'embedding-generation', 'email'
- Job: a unit of work — serialised data added to the queue
- Worker: the consumer — pulls jobs from queue, runs the processor function
- Concurrency: how many jobs a single worker processes in parallel (default: 1)
- Backoff strategies: exponential backoff for OpenAI API 429 retries
- Job priority: CRITICAL > HIGH > NORMAL > LOW — process important jobs first

AI Document Processing Pipeline Design

- Stage 1 — Upload job: validate file, store in S3, create DB record with status='pending'
- Stage 2 — Parse job: extract text from PDF/DOCX, clean whitespace, detect language
- Stage 3 — Chunk job: split text into overlapping chunks (512 tokens, 50 token overlap)
- Stage 4 — Embed job: send chunks to OpenAI Embeddings API, store vectors in Qdrant
- Stage 5 — Index job: update DB status to 'ready', notify user via SSE/email
- Parent/child jobs in BullMQ: chain these stages so each waits for the previous

> ⍰ Each stage should be idempotent: if Stage 3 runs twice, the output is the same. Store stage completion status in the database. On retry, skip stages that already succeeded.

Error Handling & Dead-Letter Queue

- Max attempts: 3 retries with exponential backoff (1s, 2s, 4s)
- Failed jobs go to the 'failed' queue — visible in Bull Board dashboard
- Webhook notification: alert user when their document processing fails
- Dead-letter queue: failed jobs older than 24h get archived for manual review

7.3 — Stop Conditions

☐ Build a 5-stage document processing pipeline with BullMQ parent/child jobs
☐ Implement idempotent job handlers — running a job twice produces the same database state
☐ Implement exponential backoff retry for OpenAI API failures (rate limit errors)
☐ Set up Bull Board monitoring dashboard — all queues visible with job counts and failure rates
☐ Load test: add 1000 jobs to the queue — all processed correctly with no data loss
☐ Implement a dead-letter queue — failed jobs older than 24h archived, user notified via email

TOPIC 8 — API Security (OWASP, Rate Limiting, Input Validation) · *Week 8 · Security is not optional for AI products*

AI APIs are attacked constantly. A single security bug can expose user data, drain your OpenAI API credits, or let attackers use your AI for free. This topic covers the OWASP API Security Top 10 and the specific security patterns needed for AI SaaS.

8.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	OWASP API Security Top 10 — Full Explanation (1 hr)	Hussein Nasser	YouTube → Search 'OWASP API Security Top 10 Full Explanation Hussein Nasser'	Most comprehensive OWASP API security video on YouTube. Cover this before anything else.
2	Video	Node.js Security Best Practices — Production Checklist	Traversy Media	YouTube → Search 'Node.js Security Best Practices Production Checklist Traversy Media'	Practical security checklist. Covers Helmet, CORS, input validation, env vars.
3	Video	Rate Limiting in Node.js — Multiple Strategies	Hussein Nasser	YouTube → Search 'Rate Limiting Node.js Multiple Strategies Hussein Nasser'	Covers token bucket, leaky bucket, sliding window — all the algorithms you need.
4	Video	SQL Injection — How It Works and How to Prevent It	Web Dev Simplified	YouTube → Search 'SQL Injection How It Works Prevent Web Dev Simplified'	Prisma parameterises queries by default, but you still need to understand this attack.
5	Video	CORS Explained — How to Fix CORS Errors	Fireship	YouTube → Search 'CORS Explained How to Fix CORS Errors Fireship'	Clear CORS explanation. Misconfigured CORS is in the OWASP top 10.
6	Video	Helmet.js — Security Headers for Express/Fastify	The Coding Garden	YouTube → Search 'Helmet.js Security Headers Express Fastify The Coding Garden'	Security headers prevent XSS, clickjacking, MIME sniffing attacks.
7	Video	Environment Variables and Secrets Management — Best Practices	Theo (t3.gg)	YouTube → Search 'Environment Variables Secrets Management Best Practices Theo t3gg'	How to handle API keys, database credentials, and secrets in production.
8	Video	CSRF Attacks — How They Work and How to Prevent Them	Hussein Nasser	YouTube → Search 'CSRF Attacks How They Work Prevent Hussein Nasser'	CSRF is relevant when your AI SaaS has cookie-based auth.
9	Guide	OWASP API Security Top 10 (2023 edition)	OWASP Foundation	owasp.org/www-project-api-security — download the PDF	Read the full OWASP API Top 10 PDF. Bookmark it. Review it before every major release.

8.2 — What to Study

OWASP API Security Top 10 (memorise these)

- API1 — Broken Object Level Authorization (BOLA): user A can access user B's conversations by guessing IDs. Fix: always filter by org_id + user_id in every query.
- API2 — Broken Authentication: weak JWT secrets, no token expiry, tokens in URLs. Fix: RS256, short TTLs, HttpOnly cookies for refresh tokens.
- API3 — Broken Object Property Level Auth: endpoint returns full DB record including internal fields. Fix: always use a DTO (Data Transfer Object) to whitelist response fields.
- API4 — Unrestricted Resource Consumption: no rate limiting → attacker runs 10,000 GPT-4o requests. Fix: per-user rate limits tied to plan tier + Redis counters.
- API5 — Broken Function Level Auth: admin endpoints accessible to regular users. Fix: RBAC middleware on every admin route.
- API8 — Security Misconfiguration: CORS *, verbose error messages, debug endpoints in production.
- API10 — Unsafe Consumption of APIs: trusting LLM output and inserting it directly into SQL or shell commands. Fix: always sanitise LLM outputs.

Input Validation — Never Trust User Input

- Validate ALL request inputs with Zod schema before processing
- Max length on message content — prevent 100,000-token prompt injections
- File type validation for uploads — check magic bytes, not just file extension
- Sanitise filenames — path traversal: ../../etc/passwd as filename
- Never pass user input directly to SQL, shell commands, or file system paths

Security Headers with Helmet.js

- Content-Security-Policy: prevent XSS by restricting script sources
- X-Frame-Options: DENY — prevent clickjacking
- Strict-Transport-Security: force HTTPS
- X-Content-Type-Options: nosniff — prevent MIME sniffing
- Referrer-Policy: no-referrer — don't leak URLs to third parties

8.3 — Stop Conditions

☐ Perform a self-audit against OWASP API Top 10 — document findings and mitigations
☐ Verify BOLA protection: attempt to access another org's conversations — server returns 403
☐ Implement input validation: request with 100,000-character message is rejected with 422
☐ Configure Helmet.js: verify all security headers present in production responses
☐ Rate limiting: verify per-user limits work — 21st request/minute returns 429
☐ Verify no sensitive data (API keys, passwords, full user objects) appears in API responses or logs

TOPIC 9 — System Design Fundamentals · *Week 9 · Think like an architect, not just a coder*

System design skills separate junior engineers from senior ones. At ₹30L+ salary levels, you will be asked to design systems in interviews and on the job. Start building this mental muscle now — it compounds over time. Focus on AI system design patterns specifically.

9.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Playlist	System Design for Beginners (playlist, 15 videos)	Gaurav Sen	YouTube → Search 'System Design Beginners Gaurav Sen' → open playlist	The best free system design YouTube series. Covers load balancing, caching, databases, consistency.
2	Playlist	System Design Interview (playlist, 20 videos)	Exponent	YouTube → Search 'System Design Interview Exponent' → open playlist	Step-by-step framework for answering system design questions. Great structure.
3	Video	Design a Chat System like WhatsApp — System Design	Gaurav Sen	YouTube → Search 'Design Chat System WhatsApp System Design Gaurav Sen'	Directly relevant: designing a chat system. The AI chat is an evolution of this.
4	Video	Design a URL Shortener — Complete System Design	Gaurav Sen	YouTube → Search 'Design URL Shortener Complete System Design Gaurav Sen'	Classic system design problem. Covers consistent hashing, database sharding.
5	Video	Horizontal vs Vertical Scaling — Complete Guide	Gaurav Sen	YouTube → Search 'Horizontal vs Vertical Scaling Complete Guide Gaurav Sen'	Fundamental scaling concept. Know when to scale up vs scale out.
6	Video	CAP Theorem Simplified — Consistency, Availability, Partition Tolerance	Gaurav Sen	YouTube → Search 'CAP Theorem Simplified Gaurav Sen'	CAP theorem is asked in every senior system design interview.
7	Video	Design an AI Document Processing System	ByteByteGo	YouTube → Search 'Design AI Document Processing System ByteByteGo'	This is the system you are building in Phase 2–3. Watching this WHILE building it is powerful.
8	Video	Designing a RAG System at Scale — Architecture Deep Dive	Weaviate (official channel)	YouTube → Search 'Designing RAG System at Scale Architecture Weaviate'	How to architect a production RAG system — the core of Phase 4.
9	Book	System Design Interview Vol 1 — Alex Xu	Alex Xu	amazon.com or PDF (widely available) — read Chapter 1, 2, 5 (consistent hashing), 6 (key-value store)	Best system design book. Chapters 1–8 are essential. Read during Month 4–5.

9.2 — AI System Design Patterns

Design Framework (use in every interview)

- Step 1: Clarify requirements — functional (what it does) + non-functional (scale, latency, availability)
- Step 2: Estimate scale — DAU, requests/second, storage requirements
- Step 3: High-level design — components and how they connect
- Step 4: Deep-dive — focus on the 2–3 hardest components
- Step 5: Identify and address bottlenecks and trade-offs

Key System Design Concepts for AI Engineers

- Load balancing: round-robin, least connections, consistent hashing
- Caching layers: CDN → Nginx → Redis → Application → Database
- Database: read replicas for scale, write master for consistency
- Message queues: async decoupling — producer doesn't wait for consumer
- API gateway: authentication, rate limiting, routing — before your service
- CDN: static assets, cached AI responses, geographic distribution

⍰ Practice: Design the AI SaaS you are building right now as if it needs to handle 1 million users. What breaks first? (Answer: the LLM API calls — you need intelligent caching and model routing.) This is the most useful design exercise.

9.3 — Stop Conditions

☐ Design an AI chat system handling 1M daily active users — whiteboard diagram with all components
☐ Identify the 5 bottlenecks in your current AI SaaS design and propose solutions for each
☐ Design a RAG document search system — from upload to query response — all components
☐ Explain CAP theorem in context of your AI database choices (Postgres + Redis + Qdrant)
☐ Calculate: if your AI app processes 10,000 conversations/day at 1,500 tokens each, what is monthly OpenAI cost?

TOPIC 10 — Stripe: Usage-Based Billing for AI SaaS · *Week 10 · Token-based billing is the AI SaaS revenue model*

AI SaaS monetisation is almost universally usage-based — you pay for what you consume (tokens, API calls, documents processed). Stripe's usage-based billing with meters handles this elegantly. Getting billing right early means you can launch a paid product by the end of Phase 5.

10.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Stripe Full Tutorial for SaaS — Subscriptions, Webhooks, Billing Portal (2 hrs)	Theo (t3.gg)	YouTube → Search 'Stripe Full Tutorial SaaS Subscriptions Webhooks Theo t3gg'	The most comprehensive Stripe tutorial for SaaS developers. Watch this completely.
2	Video	Stripe Webhooks — The Complete Guide	Web Dev Simplified	YouTube → Search 'Stripe Webhooks Complete Guide Web Dev Simplified'	Webhooks are the most error-prone part of Stripe integration. This video fixes that.
3	Video	Usage-Based Billing with Stripe Meters — Full Tutorial	Stripe (official Dev Advocates)	YouTube → Stripe official YouTube → Search 'Usage Based Billing Stripe Meters'	Token-based billing for AI. New Stripe Meters API is perfect for token consumption billing.
4	Video	Stripe Customer Portal — Subscription Management UI	James Perkins	YouTube → Search 'Stripe Customer Portal Subscription Management James Perkins'	Self-service plan management — users upgrade/downgrade without your support team.
5	Video	Stripe in Next.js App Router — Complete Integration	Josh tried coding	YouTube → Search 'Stripe Next.js App Router Complete Integration Josh tried coding'	Specifically for Next.js App Router. Follow along to build the billing integration.
6	Video	Failed Payment Handling — Stripe Dunning and Recovery	CJ Avilla (Stripe Developer Advocate)	YouTube → Search 'Failed Payment Handling Stripe Dunning CJ Avilla'	Revenue recovery: how to handle failed payments without losing users.
7	Docs	Stripe Official Documentation — Usage-Based Billing	Stripe Team	stripe.com/docs/billing/subscriptions/usage-based — read the complete guide	Stripe's docs are excellent. Read the entire Usage-Based Billing section and Webhooks guide.

10.2 — What to Study

Stripe Core Concepts

- Customer: represents a user/org in Stripe — linked to your database org_id
- Product: what you sell — 'AI SaaS Starter', 'AI SaaS Pro'

- Price: how you charge — flat monthly + usage overage
- Subscription: ongoing billing relationship between customer and product
- Invoice: monthly bill generated from subscription + usage records
- Payment Intent: handles PCI-compliant card processing

Usage-Based Billing Architecture for AI SaaS

- Plan tiers: FREE (10K tokens/month), STARTER ₹999/mo (100K tokens), PRO ₹3999/mo (1M tokens)
- Token metering: after each AI API call, record token usage in Stripe Meter via API
- Overage billing: tokens above plan limit billed at ₹0.10 per 1K tokens
- Usage record: POST /v1/billing/meter_events — call this after every AI request
- Billing portal: hosted UI — users manage their subscription without your UI work

Webhooks — Handle Every Event

- customer.subscription.created — provision user access, update DB plan_tier
- customer.subscription.updated — handle plan upgrades/downgrades
- customer.subscription.deleted — downgrade to FREE, retain data for 30 days
- invoice.payment_failed — send payment failure email, grace period 3 days
- invoice.payment_succeeded — send receipt, log revenue
- ALWAYS verify webhook signature using Stripe-Signature header + stripe.webhooks.constructEvent()

> ⍰ Webhook idempotency: webhooks can be delivered multiple times. Use Stripe event IDs to deduplicate — store processed event IDs in your database. Handling the same event twice should be safe.

10.3 — Stop Conditions

☐ Implement 3-tier plan billing: FREE, STARTER, PRO with different token limits enforced in middleware
☐ Usage metering: token consumption recorded in Stripe Meter after every AI API call
☐ Webhook handler: subscription events correctly update org plan_tier in your database
☐ Failed payment: user receives email, access continues for 3-day grace period, then downgraded
☐ Billing portal: user can upgrade, downgrade, cancel, and update payment method without contacting support
☐ Webhook idempotency: processing the same Stripe event twice produces the same database state

⍰ Phase 2 — Week-by-Week Schedule (12 Weeks)

Follow this schedule precisely. Each week has one primary topic, specific videos to watch, a concrete hands-on task, and a measurable done condition.

Week	Topic Focus	Exact Videos to Watch	Hands-On Task	Done When...
Wk 1	Node.js Internals	Node.js Full Course — Dave Gray Node.js Event Loop — Hussein Nasser Node.js Streams — Hussein Nasser	Build raw HTTP server (no framework) Stream a 50MB file through a processor	Can explain event loop phases; stream works without loading file into memory
Wk 2	Node.js Advanced + Fastify Basics	Node.js Cluster + Workers — Hussein Nasser Fastify Tutorial — Traversy Media Fastify vs Express — Fireship	Migrate raw HTTP server to Fastify Add JSON Schema validation on all routes	Fastify server running with full TypeScript + Zod validation
Wk 3	REST API Design + PostgreSQL Setup	REST API Design Best Practices — Web Dev Simplified PostgreSQL Full Course — freeCodeCamp Prisma ORM Complete Tutorial — Lama Dev	Design 7-table AI SaaS schema Run migrations with Prisma	Schema created; all relations correct; can query with Prisma Client
Wk 4	PostgreSQL Advanced + Prisma Patterns	PostgreSQL Indexes — Hussein Nasser PostgreSQL EXPLAIN ANALYZE — Hussein Nasser Prisma Transactions — Prisma official	Add indexes to all foreign keys + common query columns Build Prisma transaction for message creation	EXPLAIN ANALYZE shows index scans (not seq scans) for conversation queries
Wk 5	Redis + Authentication	Redis Crash Course — Traversy Media BullMQ Tutorial — Lama Dev (first 45 min) JWT Authentication — Web Dev Simplified	Add LLM response caching with Redis Implement JWT auth with Fastify plugin	Cache hit returns in <1ms; JWT auth working with org_id claim
Wk 6	Authentication (Advanced) + SSE Streaming	Multi-Tenancy Authentication — Theo t3gg API Key Authentication — Theo t3gg Server-Sent Events Ultimate Guide — Hussein Nasser	API key auth for B2B access Raw SSE endpoint streaming OpenAI tokens	API keys work; SSE streams tokens without Vercel AI SDK abstraction
Wk 7	Background Jobs + File Processing	BullMQ Complete Tutorial — Lama Dev Background Jobs in Next.js — Theo t3gg Document Processing Pipeline — Jack Herrington	Build 5-stage document processing pipeline with BullMQ Add Bull Board monitoring dashboard	Pipeline processes PDFs end-to-end; Bull Board shows all job states
Wk 8	API Security	OWASP API Security Top 10 — Hussein Nasser Node.js Security Best Practices — Traversy Media Rate Limiting Node.js — Hussein Nasser	Self-audit against OWASP Top 10 Add Helmet.js + input validation + rate limiting	All OWASP top 10 checks pass; rate limiting verified with load test
Wk 9	System Design	System Design Beginners playlist — Gaurav Sen (first 5 videos) Design Chat System — Gaurav Sen Design AI Document Processing — ByteByteGo	Whiteboard design: AI chat system for 1M users Identify bottlenecks in your own backend	30-minute design session produces a complete architecture diagram
Wk 10	Stripe Billing	Stripe Full Tutorial SaaS — Theo t3gg Usage-Based Billing Stripe	Implement 3-tier billing with usage metering All	Test mode: upgrade, downgrade, failed payment all work correctly

Week	Topic Focus	Exact Videos to Watch	Hands-On Task	Done When...
		Meters — Stripe official Stripe Webhooks Complete Guide — Web Dev Simplified	webhook events handled correctly	
Wk 11	Integration + Testing	Fastify Testing — official docs walkthrough Playwright E2E for API endpoints	Write integration tests for all API routes E2E test for full chat + billing flow	All routes tested; no tests call real OpenAI or Stripe APIs
Wk 12	Capstone Build + Deploy + Open Source	Review deployment docs: Railway + Docker Compose	Complete ai-saas-starter Docker Compose local setup Deploy to Railway Write README	Live API on Railway; one-command local setup; detailed README on GitHub

⍰ Phase 2 Capstone Project — AI SaaS Starter Platform

This is Portfolio Project #2. Open-source it as 'ai-saas-starter' on GitHub. A free, production-quality SaaS boilerplate consistently gets 500–1000 GitHub stars when posted on Hacker News. This single project can generate more job interview requests than anything else you build in Year 1.

Full Project Specification

Project Name	ai-saas-starter (GitHub repo name — chosen for search discoverability)
Target Stars	500+ GitHub stars (post on HN 'Show HN' — free boilerplates always go viral)
Backend Stack	Node.js + Fastify + TypeScript (strict) + Prisma + PostgreSQL + Redis + BullMQ
Frontend Stack	Next.js 14 App Router + Tailwind + Shadcn/UI (integrates with Phase 1 project)
Auth	Clerk (user-facing) + Custom API key auth (B2B) — both supported simultaneously
Billing	Stripe usage-based billing — 3 plan tiers with token metering
Queues	BullMQ + Redis — document processing, email sending, webhook delivery
AI Integration	OpenAI + Anthropic adapters behind a unified interface — model-agnostic
Deployment	Railway (one-click deploy button in README) + Docker Compose (local dev)
Monitoring	Pino structured logging + Sentry error tracking + PostHog product analytics

Required Features — Build All of These

Core Infrastructure

- Fastify server with plugin architecture: auth plugin, database plugin, Redis plugin, AI plugin
- Database: full multi-tenant schema with Row Level Security enabled
- Redis: LLM response caching + session storage + rate limiting counters
- Background jobs: BullMQ queue with Bull Board monitoring UI at /admin/queues
- Graceful shutdown: drains in-flight requests + closes DB connection + empties queues

Authentication

- Clerk OAuth: Google + GitHub + email magic link
- Multi-tenancy: users create/join organisations, organisation-scoped API access
- RBAC: owner, admin, member roles with different permissions
- API key management UI: create, view (once), rotate, and revoke keys
- Feature flags: plan-based feature gates enforced at middleware level

Billing

- 3 plan tiers: FREE (10K tokens/mo), STARTER ₹999/mo (100K tokens), PRO ₹3999/mo (unlimited)
- Usage metering: every AI API call records token consumption in Stripe Meter
- Usage dashboard: org admin can see daily/monthly token consumption and cost
- Stripe Customer Portal: self-service upgrade/downgrade/cancel
- Webhook handling: subscription events update database within 10 seconds

AI Features

- Multi-provider AI adapter: OpenAI, Anthropic, Google — unified interface
- LLM response cache: exact-match cache with Redis, 40%+ cache hit rate target
- Streaming endpoint: SSE-based token streaming with reconnection support
- Document upload: multipart file handling → BullMQ pipeline → stored in S3
- Usage enforcement: requests rejected when plan token limit exceeded

Developer Experience (what makes it get stars)

- README: one-command setup ('docker compose up') — verified to work in <5 minutes
- Architecture diagram: shows all components and their connections
- Environment variables: .env.example with every variable documented
- Seed data: 'npm run seed' creates demo org, users, conversations, and sample data
- API docs: Swagger UI at /api/docs — auto-generated from Fastify schemas
- Testing: Vitest integration tests, all routes tested, no real external API calls needed
- Deployment guide: step-by-step Railway deployment + production checklist

GitHub README Must Include

- 'Deploy to Railway' one-click button at the very top
- Screenshot of the admin dashboard with real data
- Architecture diagram (components + connections)
- Full tech stack table with links to each library's docs
- Local development setup: works in <5 minutes (tested from scratch)
- Feature checklist: every implemented feature clearly listed
- 'What this is NOT' section: set honest expectations, prevent wrong-use issues

Service Architecture Diagram (Text Version)

```
┌──────────────────────────────────────────────────────────────┐
│                        CLIENTS                               │
│  Next.js App (Vercel)        │    External API (via API Key)  │
└──────────────┬──────────────┴──────────────┬──────────────────┘
               │                             │
               ▼                             ▼
┌──────────────────────────────────────────────────────────────┐
│                  Fastify API Server (Railway)                 │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Middleware: Auth (Clerk JWT / API Key) → Rate Limit  │    │
│  │            (Redis) → Plan Gate → Org Context         │    │
```

```
│  └──────────────────────────────────────────────────────────┘   │
│  Routes: /api/v1/chat  /api/v1/documents  /api/v1/conversations │
└────────┬──────────────┬──────────────┬──────────────┬──────────┘
         │              │              │              │
         ▼              ▼              ▼              ▼
   PostgreSQL       Redis          BullMQ         LLM APIs
   (Railway)        (Railway)      Workers        (OpenAI/
   Prisma ORM       Cache+Sessions Doc Pipeline    Anthropic)
                    Rate Limiting  Embed Pipeline
                                   Email Queue
```

Estimated Time Investment

Weeks 1–2 (Node.js + Fastify)	~20 hrs — 12 hrs videos + 8 hrs building
Weeks 3–4 (PostgreSQL + Prisma)	~22 hrs — 10 hrs videos + 12 hrs schema + queries
Week 5–6 (Redis + Auth)	~20 hrs — 10 hrs videos + 10 hrs implementation
Week 6 (SSE Streaming)	~10 hrs — 4 hrs videos + 6 hrs building
Week 7 (Background Jobs)	~16 hrs — 8 hrs videos + 8 hrs pipeline
Week 8 (Security)	~12 hrs — 6 hrs videos + 6 hrs implementing + auditing
Week 9 (System Design)	~10 hrs — 6 hrs videos + 4 hrs practice design
Week 10 (Stripe)	~14 hrs — 6 hrs videos + 8 hrs integration
Weeks 11–12 (Integration + Capstone)	~24 hrs — testing + deploy + README + open source
Total Phase 2	~148 hours over 12 weeks ≈ 2.5 hrs/weekday + 5 hrs/weekend

Phase 2 Completion Check: You have completed Phase 2 when: (1) ai-saas-starter is live on Railway with a working API, (2) Docker Compose local setup works in under 5 minutes, (3) Stripe test mode shows successful billing flows, (4) The GitHub README is detailed enough that a developer can deploy it without asking you questions. At this point you are genuinely full-stack.

Do NOT move to Phase 3 until the capstone project is deployed AND open-sourced. A half-built backend that exists only on localhost does not count. The GitHub repo and Railway deployment are the deliverables — not the code itself.